Travlr Getaways

CS 465 Project Software Design Document

Version 1.0

Table of Contents

CS 465 Project Software Design Document 1
Table of Contents 2
Document Revision History 2
Instructions 2
Executive Summary 3
Design Constraints 3
System Architecture View 3
Component Diagram 3
Sequence Diagram 4
Class Diagram 4
API Endpoints 4
The User Interface 4

Document Revision History

Version	Date	Author	Comments
1.0	03/26/2026	Sarah Theeb	Initial Draft of Software Design Document
1.1	04/03/2026	Sarah Theeb	Created sequence diagram, class diagram, and API endpoints
1.2	04/22/2026	Sarah Theeb	Final Project

Executive Summary

The Travlr Getaways web application will be developed using a full stack architecture based on the MEAN stack, which includes MongoDB, Express.js, Angular, and Node. This design is appropriate because it enables efficient communication between the front end and back end via JavaScript across all application layers. It enables scalability, flexibility, and real-time data handling, all of which are required for a trip booking platform to accommodate several users at the same time.

The program will have two main components: a customer-facing website and an administrative single-page application (SPA). The customer-facing website will allow visitors to browse vacation packages, read extensive destination information, and interact with available options. This section of the program will prioritize usability, accessibility, and a visually appealing interface to improve the user experience.

The administrator SPA will be designed for internal use, allowing administrators to review trip listings, change information, and keep the system running smoothly. Because it is a single-page application, it will load material faster and more dynamically, eliminating the need to refresh the full page. These components will work together to build a fully functional, efficient, and scalable full stack web application that matches the client's requirements.

Design Constraints

Several constraints on design must be considered when creating the Travlr Getaways online application. One key constraint is the requirement to use the MEAN stack, which limits the number of technologies

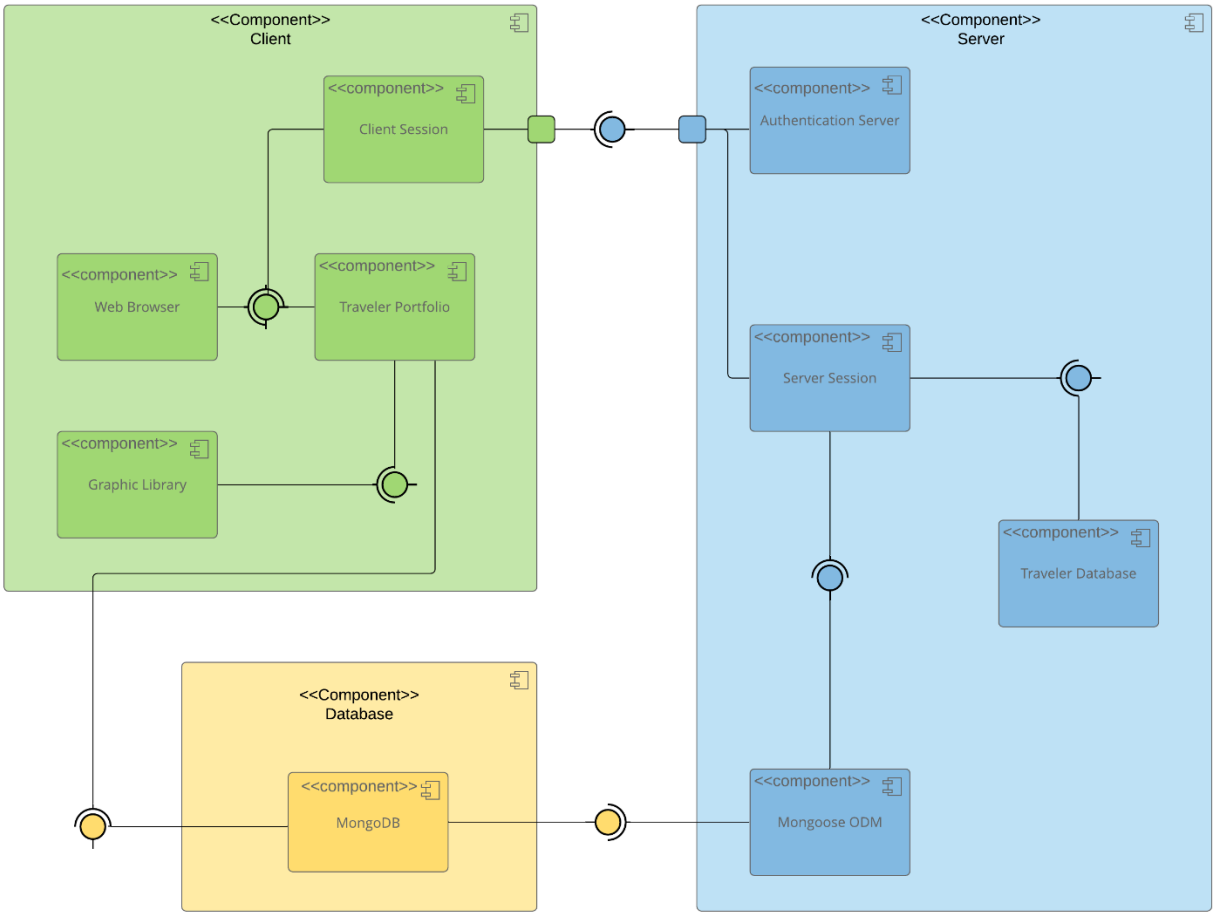
available yet assures consistency throughout development. While this has benefits such as consistent language usage, it may limit flexibility in selecting different tools or frameworks.

Another constraint is performance and scalability. The application must be able to manage several users concurrently without slowing down, which necessitates efficient database design and optimum server performance. Furthermore, security is a critical limitation because the program must safeguard user data and limit access to administrative capabilities via authentication and permission systems.

Time constraints and project needs also influence development. The program must be completed within a set time window while satisfying all functional requirements, which may limit the opportunity to include advanced features. These limits necessitate careful planning and prioritization to guarantee that the system's most vital components are created efficiently while preserving quality and performance.

System Architecture View

Component Diagram



The Travlr Getaways web application is built on a full stack architecture, which divides the system into three independent layers: client-side, server-side, and data-tier components. Each component has a

particular purpose in handling user interactions, processing requests, and storing data, ensuring a clear separation of concerns while also improving scalability and maintenance.

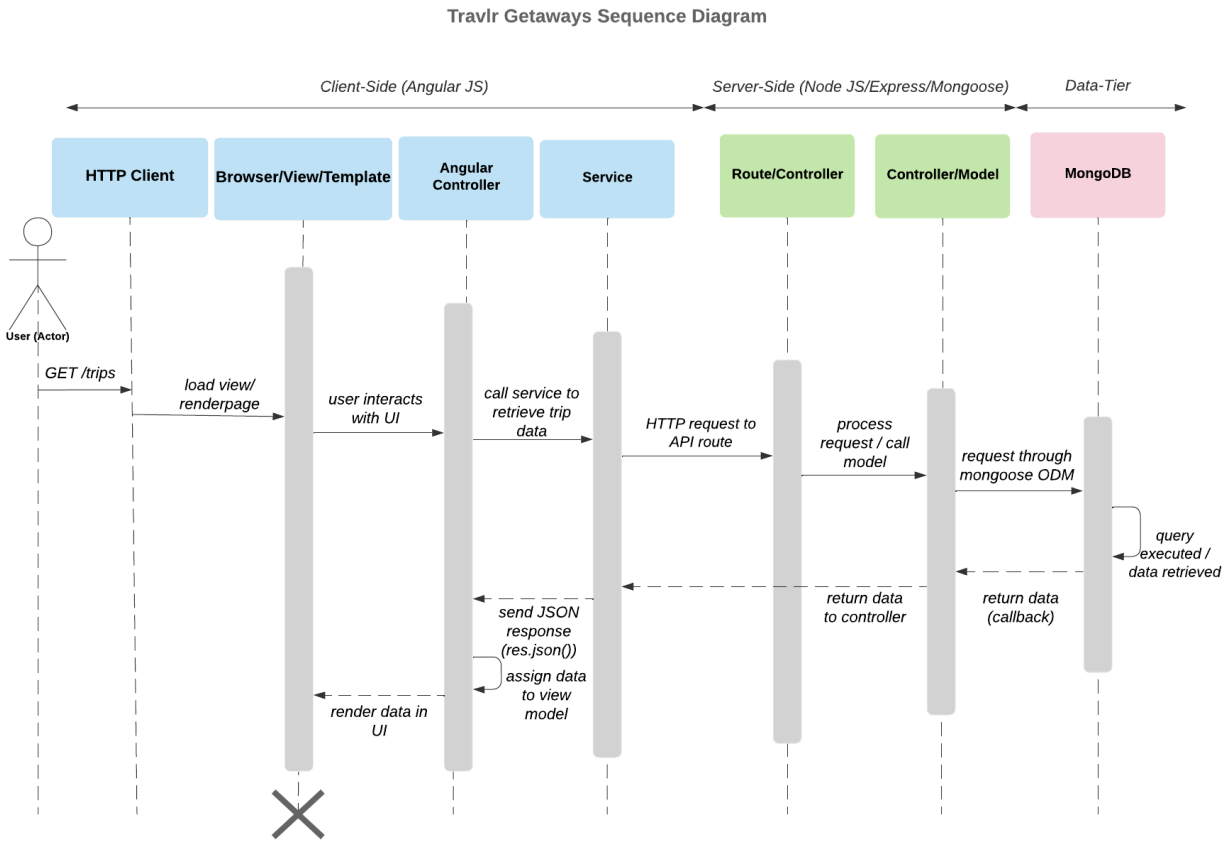
On the client side, the application includes both a customer-facing website and an administrative single-page application (SPA) created with Angular. These components are in charge of presenting the user interface and managing user interactions. The browser/view/template generates dynamic content, whereas the Angular controllers and services handle user input and send HTTP requests to the backend. This layer guarantees that users can effectively browse trips, interact with features, and take actions like viewing or managing bookings.

On the server, application logic and routing are handled using Node.js and Express.js. The Route component is in charge of receiving incoming HTTP requests and forwarding them to the appropriate controller. The Controller handles the request, applies business logic, and decides how to react. It works with the Model layer, which is in charge of managing data operations and connecting with the database. This structure makes sure that requests are handled effectively and consistently.

MongoDB manages the data tier, where all application data is stored, such as travel details, user accounts, and booking information. The application interacts with MongoDB using Mongoose ODM, which allows the model layer to execute structured queries, updates, and data validation. MongoDB handles these requests and routes the data back to the model and controller levels.

These components work together to form a cohesive system, with the client sending requests, the server processing logic and retrieving data, and the database storing and returning information. This architecture promotes efficient communication across all tiers, ensures data consistency, and enables the program to scale as user demand grows.

Sequence Diagram



The sequence diagram illustrates the complete flow of data through the Travlr Getaways full stack application, emphasizing the relationships of the client-side, server-side, and data-tier components. The process begins when the user (actor) selects the trips page in the browser. This action causes the HTTP client to submit a GET request to the application, especially for the trips route. The browser/view/template then loads and renders the first page, giving the user the ability to interact with the interface.

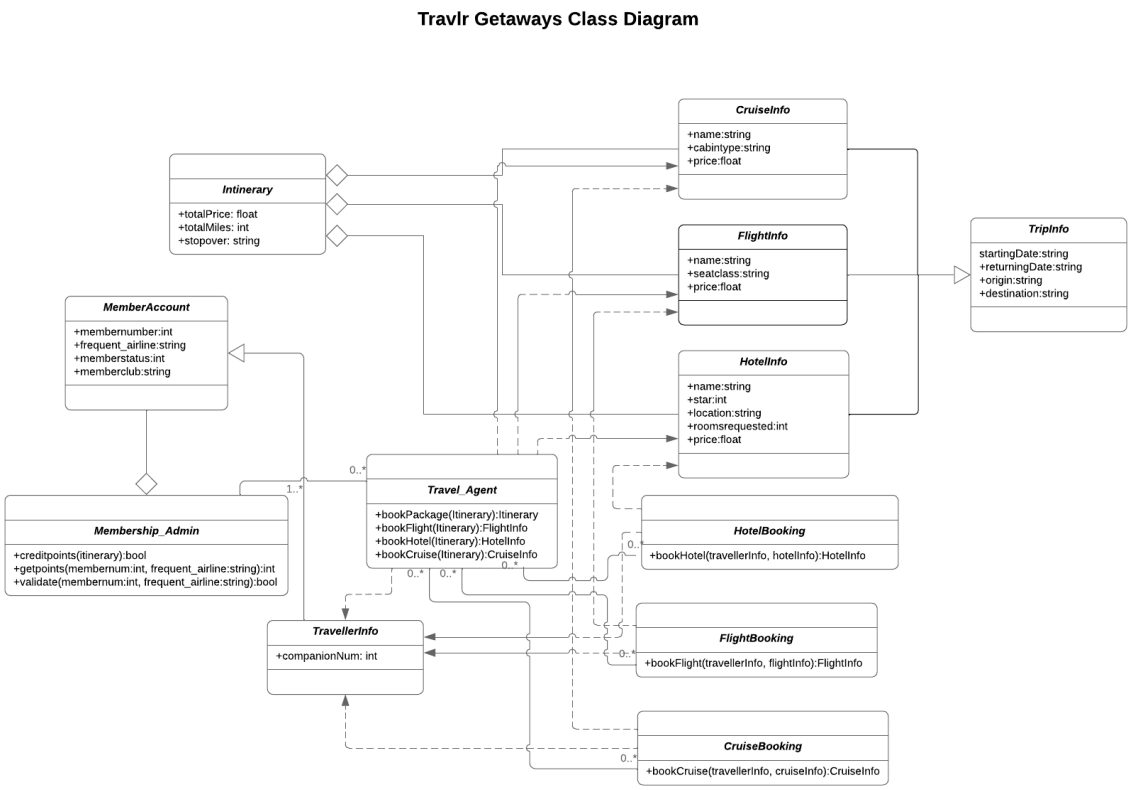
When a user interacts with the application, the Angular controller on the client side collects the event and sends it to the service layer. The service acts as an intermediary between the frontend and backend, sending HTTP requests to the server's API routes. This request is handled by the Express.js framework's Route/Controller component, which is responsible for routing incoming requests to the proper controller logic.

The controller processes the request and invokes the model layer (Controller/Model), which is responsible for performing data operations and interacting with the database. The model communicates with the MongoDB database using Mongoose ODM, which abstracts the database interactions and simplifies query execution. MongoDB processes the query, retrieves the relevant trip data, and returns the results through a callback function to the model layer.

After receiving the data, the model sends the results to the controller, which prepares the response and sends it back through the route as a JSON response. The response is sent back to the client-side service, which assigns the obtained data to the Angular view model. This allows the Angular controller to dynamically update the browser view and render the template with the retrieved data.

Finally, the browser shows the new data in the user interface, showing the user a list of possible trips. This sequence shows how each component performs a specific function in processing requests and providing data, ensuring a clear separation of responsibilities between the presentation layer, application logic, and data management. It also emphasizes how a full stack architecture promotes efficient communication, scalability, and maintainability inside the program.

Class Diagram



The class diagram illustrates the Travlr Getaways application's object-oriented structure and how its many components are arranged and interact with one another. The TripInfo class is the system's fundamental component, storing critical trip information such as the start and end dates, origin, and destination. This class is linked to supporting classes such as FlightInfo, HotelInfo, and CruiseInfo, which provide detailed information on transportation and lodging alternatives for each trip.

The Itinerary class is used as a summary of a user's chosen travel components, including total cost, total miles, and stopovers. It collects information from a variety of sources, including airplane, hotel, and cruise options, allowing the system to create a comprehensive travel itinerary. The TravellerInfo class represents the user and keeps details such as the number of companions, whereas the MemberAccount class handles user-specific data such as membership number, airline preferences, and membership status.

The FlightBooking, HotelBooking, and CruiseBooking classes handle booking functionality, which involves establishing bookings based on user selections. These classes work with both TravellerInfo and the associated service classes (FlightInfo, HotelInfo, and CruiseInfo) to guarantee that bookings are properly processed. The Travel_Agent class coordinates these booking procedures by providing methods for booking packages and particular services, and it serves as a focal point for handling user interactions with the system.

The Membership_Admin class represents administrative functionality, including methods for membership validation, point calculation, and incentive management. This ensures that membership-related aspects are handled independently of core booking logic, resulting in a clear separation of responsibilities.

Overall, the class diagram shows a logical and structured design, with each class having a well-defined responsibility. This structure increases the application's maintainability, scalability, and readability by dividing user data, booking logic, and trip management into separate components.

API Endpoints

Method	Purpose	URL	Notes
GET	Retrieve all trips	/api/trips	Returns a list of all available trips from MongoDB
GET	Retrieve a single trip	/api/trips/:tripId	Returns detailed information for a specific trip
POST	Create a new trip	/api/trips	Used by admin to add a new trip
PUT	Update an existing trip	/api/trips/:tripId	Allows admin to modify trip details
DELETE	Delete a trip	/api/trips/:tripId	Removes a trip from the database
POST	User login	/api/login	Authenticates user credentials
GET	Retrieve user profile	/api/users/:userId	Returns user account information
POST	Create booking	/api/bookings	Creates a new booking (flight, hotel, or cruise)
GET	Retrieve user bookings	/api/bookings/:userId	Returns all bookings associated with a user

The User Interface

1.

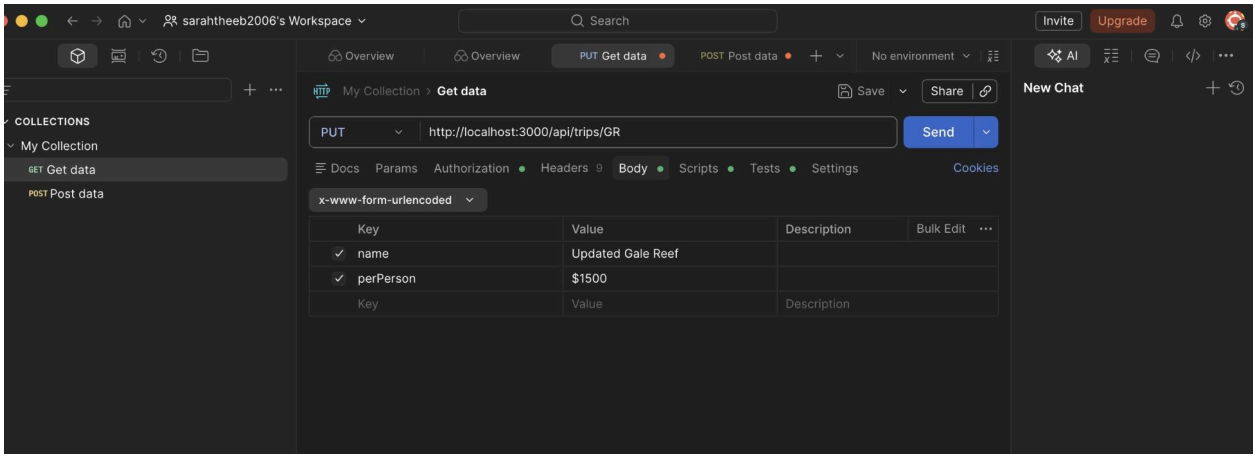
localhost:3000/api/trips

Pretty-print

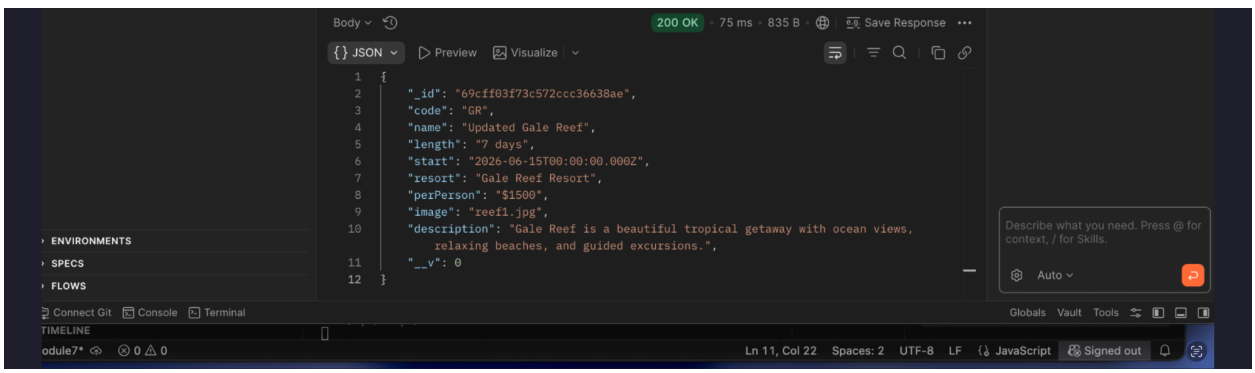

```
[
  {
    "_id": "69cff03f73c572ccc36638ae",
    "code": "GR",
    "name": "Updated Gale Reef",
    "length": "7 days",
    "start": "2026-06-15T00:00:00.000Z",
    "resort": "Gale Reef Resort",
    "perPerson": "$1500",
    "image": "reef1.jpg",
    "description": "Gale Reef is a beautiful tropical getaway with ocean views, relaxing beaches, and guided excursions.",
    "__v": 0
  },
  {
    "_id": "69cff03f73c572ccc36638af",
    "code": "DR",
    "name": "Dawson's Reef",
    "length": "5 days",
    "start": "2026-07-20T00:00:00.000Z",
    "resort": "Dawson Reef Resort",
    "perPerson": "$999",
    "image": "reef2.jpg",
    "description": "Dawson's Reef offers a shorter island escape with snorkeling, dining, and comfortable accommodations.",
    "__v": 0
  },
  {
    "_id": "69cff03f73c572ccc36638b0",
    "code": "CR",
    "name": "Claire's Reef",
    "length": "6 days",
    "start": "2026-08-10T00:00:00.000Z",
    "resort": "Claire Reef Resort",
    "perPerson": "$1199",
    "image": "reef3.jpg",
    "description": "Claire's Reef is a scenic vacation destination featuring reef tours, beach activities, and oceanfront rooms.",
    "__v": 0
  }
]
```


This screenshot shows the trip data retrieved from the API endpoint using a GET request. It includes a unique trip that was added and later updated during development, as seen by the modified trip name and price. This confirms that the application successfully communicates with the database and that changes made through the API are persisted. The ability to retrieve updated trip data demonstrates that the backend is correctly storing and returning information, which is essential for the functionality of the single-page application.

2.



This screenshot shows the process of editing an existing trip using the API. The PUT request is sent to the endpoint /api/trips/GR, where the trip with the code “GR” is being modified. The fields name and perPerson are updated to new values, demonstrating how the admin interface allows changes to existing trip data. This represents the edit functionality of the application, where user input is prepared and sent to the backend for processing.

3. 

This screenshot shows the successful update of a trip after the PUT request is executed. The response displays a 200 OK status along with the updated trip data returned from the server. The presence of the modified values, such as the updated trip name and price, confirms that the backend has processed the request and saved the changes in the database. This demonstrates that the API is functioning correctly and that updates persist in the system.

The Angular administrator interface is organized into standalone components, templates, and service logic that communicate with the Express REST API. This differs from the customer-facing Express application, which is structured using routes, controllers, views, and models. In the Express application, the user interface is rendered on the server using Handlebars templates, while the Angular single-page application dynamically updates content in the browser without requiring a full page refresh. This provides richer functionality for administrative tasks because trip data can be retrieved, displayed, and updated more interactively than in a traditional multi-page web application.

The SPA was tested by verifying that it could communicate with the API to retrieve and update trip data stored in MongoDB. GET requests were used to confirm that trip information could be returned from the /api/trips endpoint, and PUT requests were tested by updating an existing trip after authentication. The update testing confirmed that the API, database, and application logic were working together correctly because the updated values were returned in the response and reflected in the stored trip data. Authentication and security were also tested by registering a user, logging in to receive a JSON Web Token, and using that token to access protected administrative endpoints.